Final Exam: Computer Architecture, 2024 Spring

Student Exam ID: _____

Instructions:

1. **Be ready.** Please read the instructions in full
2. **Be strategic.** Exam points are graded based on 100 points - we will later translate into a 7 step scale. Try to get as many points as you can. If you are stuck, try to move to another question and collect points.
3. **Be careful.** Please read the questions carefully before responding
4. **Be thorough.** Show us your procedure. No procedure and perfect result has few points. Good procedure with minor mistakes will get most points in a question.
5. **Be smart.** Do not copy text from the Internet. We will find out and there are consequences to it.

Assembly instructions help

`movq A B`	B = A
`salq A B` `sarq A B`	B = B << A B = B >> A
`imulq A B`	B = A * B
`divq A B`	B = B / A
`subq A B`	B = B - A
`cmpq A B`	Compare A to B (used before conditional expression)
`je [label]` `jne, jl, jle, jg, jge`	Jump to label if the previous line evaluates to “equal” Other variants (in the same order as listed): not equal, less, less or equal, greater, greater or equal
`leaq Src Dest`	Compute the memory address expression Src and put the result to Dest
`pushq Src`	Push the value at Src to the top of the stack
`popq Dest`	Read the value from the top of the stack and copy it to Dest

Question 1 (25 points)

In the memory of a computer, you find 4 bytes of critical data. From address 0x00000000 to 0x00000003 you find the values 0x48, 0x45, 0x21, and 0x08, respectively, corresponding to the four bytes. We do not know the type of the 4 bytes (e.g., char, int, float) or specifics of the system (e.g., big-endian, little-endian). We will proceed to analyze the main options.

1.1) Please define big-endian (2 points):

1.2) Please define little-endian (2 point):

1.3) Describe how do you calculate the value of an **int** (3 points)

1.4) Assuming the value is a 32-bit **int**, what is the value of it if the system is big-endian or little-endian? (6 points)

Big-endian value:

Little-endian value:

1.5) If you proceeded to **cast** the 32-bit **int** into a **float**, would you expect any loss of information? Why or why not? (3 points)

1.6) Repeat problem 1.5 but consider that you cast the 32-bit **int** into a **double**. (3 points)

1.7) If we assume it is a **char** array, what does the encoded message read? Please use the ASCII table below as reference. (6 points)

ASCII TABLE

Decimal	Hexadecimal	Binary	Octal	Char	Decimal	Hexadecimal	Binary	Octal	Char	Decimal	Hexadecimal	Binary	Octal	Char
0	0	0	0	*[NULL]*	48	30	110000	60	**0**	96	60	1100000	140	`
1	1	1	1	*[START OF HEADING]*	49	31	110001	61	**1**	97	61	1100001	141	**a**
2	2	10	2	*[START OF TEXT]*	50	32	110010	62	**2**	98	62	1100010	142	**b**
3	3	11	3	*[END OF TEXT]*	51	33	110011	63	**3**	99	63	1100011	143	**c**
4	4	100	4	*[END OF TRANSMISSION]*	52	34	110100	64	**4**	100	64	1100100	144	**d**
5	5	101	5	*[ENQUIRY]*	53	35	110101	65	**5**	101	65	1100101	145	**e**
6	6	110	6	*[ACKNOWLEDGE]*	54	36	110110	66	**6**	102	66	1100110	146	**f**
7	7	111	7	*[BELL]*	55	37	110111	67	**7**	103	67	1100111	147	**g**
8	8	1000	10	*[BACKSPACE]*	56	38	111000	70	**8**	104	68	1101000	150	**h**
9	9	1001	11	*[HORIZONTAL TAB]*	57	39	111001	71	**9**	105	69	1101001	151	**i**
10	A	1010	12	*[LINE FEED]*	58	3A	111010	72	**:**	106	6A	1101010	152	**j**
11	B	1011	13	*[VERTICAL TAB]*	59	3B	111011	73	**;**	107	6B	1101011	153	**k**
12	C	1100	14	*[FORM FEED]*	60	3C	111100	74	**<**	108	6C	1101100	154	**l**
13	D	1101	15	*[CARRIAGE RETURN]*	61	3D	111101	75	**=**	109	6D	1101101	155	**m**
14	E	1110	16	*[SHIFT OUT]*	62	3E	111110	76	**>**	110	6E	1101110	156	**n**
15	F	1111	17	*[SHIFT IN]*	63	3F	111111	77	**?**	111	6F	1101111	157	**o**
16	10	10000	20	*[DATA LINK ESCAPE]*	64	40	1000000	100	**@**	112	70	1110000	160	**p**
17	11	10001	21	*[DEVICE CONTROL 1]*	65	41	1000001	101	**A**	113	71	1110001	161	**q**
18	12	10010	22	*[DEVICE CONTROL 2]*	66	42	1000010	102	**B**	114	72	1110010	162	**r**
19	13	10011	23	*[DEVICE CONTROL 3]*	67	43	1000011	103	**C**	115	73	1110011	163	**s**
20	14	10100	24	*[DEVICE CONTROL 4]*	68	44	1000100	104	**D**	116	74	1110100	164	**t**
21	15	10101	25	*[NEGATIVE ACKNOWLEDGE]*	69	45	1000101	105	**E**	117	75	1110101	165	**u**
22	16	10110	26	*[SYNCHRONOUS IDLE]*	70	46	1000110	106	**F**	118	76	1110110	166	**v**
23	17	10111	27	*[END OF TRANS. BLOCK]*	71	47	1000111	107	**G**	119	77	1110111	167	**w**
24	18	11000	30	*[CANCEL]*	72	48	1001000	110	**H**	120	78	1111000	170	**x**
25	19	11001	31	*[END OF MEDIUM]*	73	49	1001001	111	**I**	121	79	1111001	171	**y**
26	1A	11010	32	*[SUBSTITUTE]*	74	4A	1001010	112	**J**	122	7A	1111010	172	**z**
27	1B	11011	33	*[ESCAPE]*	75	4B	1001011	113	**K**	123	7B	1111011	173	**{**
28	1C	11100	34	*[FILE SEPARATOR]*	76	4C	1001100	114	**L**	124	7C	1111100	174	**|**
29	1D	11101	35	*[GROUP SEPARATOR]*	77	4D	1001101	115	**M**	125	7D	1111101	175	**}**
30	1E	11110	36	*[RECORD SEPARATOR]*	78	4E	1001110	116	**N**	126	7E	1111110	176	**~**
31	1F	11111	37	*[UNIT SEPARATOR]*	79	4F	1001111	117	**O**	127	7F	1111111	177	*[DEL]*
32	20	100000	40	*[SPACE]*	80	50	1010000	120	**P**					
33	21	100001	41	**!**	81	51	1010001	121	**Q**					
34	22	100010	42	**"**	82	52	1010010	122	**R**					
35	23	100011	43	**#**	83	53	1010011	123	**S**					
36	24	100100	44	**$**	84	54	1010100	124	**T**					
37	25	100101	45	**%**	85	55	1010101	125	**U**					
38	26	100110	46	**&**	86	56	1010110	126	**V**					
39	27	100111	47	**'**	87	57	1010111	127	**W**					
40	28	101000	50	**(**	88	58	1011000	130	**X**					
41	29	101001	51	**)**	89	59	1011001	131	**Y**					
42	2A	101010	52	*****	90	5A	1011010	132	**Z**					
43	2B	101011	53	**+**	91	5B	1011011	133	**[**					
44	2C	101100	54	**,**	92	5C	1011100	134	****					
45	2D	101101	55	**-**	93	5D	1011101	135	**]**					
46	2E	101110	56	**.**	94	5E	1011110	136	**^**					
47	2F	101111	57	**/**	95	5F	1011111	137	**_**					

Question 2 (25 points)

As part of your job, you are given different tasks analyzing code snippets in C and assembly and validating their operation.

2.1) Given the following assembly code, write the C code associated to it (12 points)

```
# long dummy(long n)
# n: %rdi
dummy:
    movq    $1, %rax
    jmp     .TEST

.LOOP:
    imulq   %rdi, %rax
    subq    $1, %rdi

.TEST:
    cmpq    $1, %rdi
    jg      .LOOP
    ret
```

2.2) Given the following function in C, you are asked to validate its operation. Answer the questions below (Total of 13 points, breakdown below)

```c
#include <stdio.h>
int operat(unsigned int x, unsigned int y)
{
   unsigned int t = x << 4;
   t = t - x;
   y = y >> 4
   return t+y;
}
```

- What is the final result in terms of 'x' and 'y'? *Hint: you can express it as a mathematical function. Think of the interpretation of << and >>* (5 points)
- For which values of x is your function valid? *Hint: consider UINT_MAX as the maximum representable unsigned integer.* (5 points)
- If x = 4 and y = 192, what does the function return? (3 points)

Question 3 (25 points)

This question focuses on accelerating the operation of a CPU that has been optimized for the execution of a new task. You are first asked to describe the new hardware component added, and then analyze a pipelined architecture that could exploit it.

3.1) Describe the operation of the following HCL code function describing the new hardware-accelerated function. Analyze what it achieves. As in class, assume that capitalized letters are words (e.g., integers of n bits). Otherwise, they represent booleans. (12 points)

```
int function = [    A >= B && A <= C        : A;
                    B >= A && C <= A        : A;
                    B >= A && B <= C        : B;
                    B <= A && B >= C        : B;
                    1                       : C;
 ];
```

3.2) Unfortunately adding this "function" is complex. When analyzing the way you can split the system into four pipeline stages, one of the stages will require more time to settle (assume this is Stage 1). Assume that you can only split in at most 4 stages, where Stage 1 logic takes 110 ps, Stage 2 logic takes 50 ps, Stage 3 logic takes 25 ps, and Stage 4 logic takes 25 ps. Registers in each stage add a delay of 20ps. (13 points)

- Compute the throughput for the pipeline in GIPS.
- Compute the delay for an instruction.
- If you transformed it into a 3-stage pipeline, what is the best choice of stages to merge? What is the resulting throughput and delay?
- If you transformed it into a 2-stage pipeline, what is the best choice of stages to merge? What is the resulting throughput and delay?

Question 4 (25 points)

This question considers the design of cache's and their performance under different conditions. Please answer each question clearly and explain your procedure.

For the following questions consider that the **main memory size is 64 bytes.**

4.1) What is a Stride-1 access pattern, and what type of locality does it exploit? (4 points)

4.2) Consider a directly mapped cache, a cache block size of 4 bytes, S = 4 sets.

- How many blocks per set do you have? (2 points)

  E =

- Describe the cache structure and the bits of each address used for selecting the set the tag (3 points)

- The code reads bytes through a one dimensional `char` array of 24 elements (i.e., 24 bytes). The byte array starts in address 0x000. It reads in a stride-one pattern. Calculate the cache miss ratio (5 points)

4.3) Repeat problem 4.2 considering that you change the cache block to 8 bytes and S = 2 sets.

- Describe the cache structure and the bits of each address used for selecting the set the tag (3 points)

- The code reads bytes through a one dimensional `char` array of 24 elements (i.e., 24 bytes). The byte array starts in address 0x000. It reads in a stride-one pattern. Calculate the cache miss ratio and compare to the result in problem 4.2 (5 points)

4.4) Based on the results from 4.2 and 4.3, for the case of a stride-one pattern, would it be an advantage to move to a direct-mapped cache with S = 1 set and a cache block of 16 bytes? (3 points)